asynchronous result in the `std::future` obtained from `get_future()`. You can thus wrap a task in a `std::packaged_task` and retrieve the future before passing the `std::packaged_task` object elsewhere to be invoked in due course. When you need the result, you can wait for the future to become ready. The following example shows this in action.

Passing Tasks Between Threads

Many GUI frameworks require that updates to the GUI be done from specific threads, so if another thread needs to update the GUI, it must send a message to the right thread in order to do so. `std:packaged_task` provides one way of doing this without requiring a custom message for each and every GUI-related activity, as shown here.

Listing 4.9. Running code on a GUI thread using `std::packaged_task`

```
#include <deque>
#include <mutex>
#include <future>
#include <thread>
#include <utility>

std::mutex m;
std::deque<std::packaged_task<void()> > tasks;

bool gui_shutdown_message_received();
void get_and_process_gui_message();

void gui_thread()                                   <-(1)
{
    while(!gui_shutdown_message_received())         <-(2)
    {
        get_and_process_gui_message();              <-(3)
        std::packaged_task<void()> task;
        {
            std::lock_guard<std::mutex> lk(m);
            if(tasks.empty())                       <-(4)
                continue;
            task=std::move(tasks.front());          <-(5)
            tasks.pop_front();
        }
        task();                                     <-(6)
    }
}

std::thread gui_bg_thread(gui_thread);

template<typename Func>
std::future<void> post_task_for_gui_thread(Func f)
{
    std::packaged_task<void()> task(f);             <-(7)
    std::future<void> res=task.get_future();        <-(8)
    std::lock_guard<std::mutex> lk(m);
    tasks.push_back(std::move(task));               <-(9)
    return res;                                     <-(10)
}
```

This code is very simple: the GUI thread ❶ loops until a message has been received telling the GUI to shut down ❷, repeatedly polling for GUI messages to handle ❸, such as user clicks, and for tasks on the task queue. If there are no tasks on the queue ❹, it loops again; otherwise, it extracts